

MINISTRY OF EDUCATION AND TRAINING
FPT UNIVERSITY

CAPSTONE PROJECT PROPOSAL

A Web-Based System for Real-Time Hotel Market Price Monitoring and Forecasting Using Machine Learning

By

Dang Nguyen Minh An

Student ID: 24MSE23217

Supervisor: Dr. Le Thanh Hai

A proposal submitted in conformity with the requirements
for the degree of Master of Software Engineering

© Copyright by Dang Nguyen Minh An - 2026

Abstract

Room rates on Online Travel Agencies (OTAs) move continuously under dynamic pricing, and the tools that turn this movement into pricing decisions are priced for hotel chains rather than for the independent small and medium properties that make up most of Vietnam’s accommodation supply. This proposal describes a web-based system that collects Booking.com room rates for hotels in five Vietnamese cities, forecasts short-term rate movements with machine learning, and serves those forecasts through two decision layers built on one shared model: a hotelier view centred on competitive-set monitoring and pricing-position alerts, and a consumer view centred on booking-timing advice.

The design rests on one modelling requirement that shapes the entire data pipeline. Every price record carries two independent time coordinates, the observation timestamp and the stay date, so the dataset records how the rate for a fixed check-in date evolves as that date approaches. Lead time, defined as the difference between the two, becomes the strongest single predictor available. A durable MySQL work queue drives a Selenium crawler that reuses one browser session across ten queued items, records candidate, parsed and persisted option counts for every attempt, and separates sold-out nights from properties that cannot be booked at all and from dead links. Room comparability is maintained by a reference definition that the system calibrates per hotel and per check-in date, promoting a candidate to approved status only after it appears in at least three completed runs with item coverage of at least 80 percent.

Four models representing distinct methodological families, XGBoost, Random Forest, LSTM and Transformer, will be trained for the 1, 3, 7 and 14-day horizons and compared under a strictly chronological train/validation/test split. Evaluation combines forecast accuracy (target MAPE $\leq 12\%$ at the 7-day horizon, price-direction accuracy $\geq 80\%$), SHAP-based interpretability, and two retrospective simulations that measure practical value: a pricing opportunity analysis for the hotelier view and a booking savings simulation for the consumer view.

A six-day pilot covering 10 hotels and 5 check-in dates has already produced 2,312 price observations across 300 crawl items with no parser-level data loss, and an artifact-based validation compared 51 stored observations against the HTML captured at crawl time across 473 fields without a mismatch.

Contents

Abstract	1
List of Figures	3
List of Tables	4
List of Abbreviations	5
1 INTRODUCTION AND BACKGROUND	6
1.1 Problem Description	6
1.1.1 The Vietnamese Accommodation Market and OTA Distribution . .	6
1.1.2 Dynamic Pricing and the Rate-Intelligence Gap	6
1.1.3 Research Gap	7
1.2 Research Objectives	8
1.2.1 Main Objective	8
1.2.2 SMART Goals	8
1.2.3 Research Questions	8
1.3 Scope of the Project	10
1.3.1 In Scope	10
1.3.2 Out of Scope	11
1.3.3 Deferred to Future Work	11
2 PROPOSED SOLUTION	12
2.1 Solution Description	12
2.1.1 The Two Time Coordinates	12
2.1.2 Data Collection Subsystem	13
2.1.3 Reference Room Calibration	14
2.1.4 Feature Engineering and Label Generation	15
2.1.5 Forecasting Engine and Two Decision Layers	16
2.2 Software Architecture	17
2.2.1 Component Layers	17
2.2.2 Database Schema	19
2.2.3 Data Flow	20
2.3 Technology and Tools Used	21
2.3.1 Deviations from the Approved Topic Description	22

3	IMPLEMENTATION PLAN	24
3.1	Main Development Stages	24
3.1.1	Phase 0 – Foundation (Weeks 1–2)	24
3.1.2	Phase 1 – Collection Application and Durable Worker (Weeks 2–3)	24
3.1.3	Phase 2 – Continuous Collection (Week 3 onward, in parallel)	25
3.1.4	Phase 3 – Exploratory Analysis, Features and Labels (Weeks 5–9)	25
3.1.5	Phase 4 – Model Development (Weeks 9–16)	25
3.1.6	Phase 5 – Evaluation and Interpretability (Weeks 16–19)	26
3.1.7	Phase 6 – Dashboard and Decision Simulations (Weeks 17–22)	26
3.1.8	Phase 7 – Thesis and Defence (Weeks 20–24)	26
3.2	Expected Schedule	27
3.3	Current Implementation Status	27
3.4	Feasibility Assessment	29
3.4.1	Risks and Mitigations	29
3.4.2	Resource Requirements	30
3.4.3	Ethical Considerations	30
4	EXPECTED RESULTS	31
4.1	Deliverables	31
4.2	Performance Targets	32
4.3	Contributions	33
5	EVALUATION PLAN	34
5.1	Forecast Accuracy	34
5.1.1	Regression Metrics	34
5.1.2	Direction Classification	35
5.1.3	Baselines	35
5.2	Interpretability	35
5.3	Decision Value	36
5.3.1	Pricing Opportunity Analysis (Hotelier View)	36
5.3.2	Booking Savings Simulation (Consumer View)	36
5.4	Data and System Quality	37
5.4.1	Data Quality	37
5.4.2	System Performance	37
5.5	Threats to Validity	37
	References	39

List of Figures

2.1	Observation grid. Each dot is one crawl of one hotel for one stay date. A column is a single rate series observed at successive times; the horizontal distance from an observation to its stay date is the lead time.	13
2.2	System architecture. The API and the crawl worker are separate processes coordinating only through the MySQL queue, so neither depends on the other staying alive.	19
3.1	Project timeline over 24 weeks, August 2026 to late January 2027. Collection runs continuously alongside every phase after week 3.	27

List of Tables

1.1	SMART goal definition	9
1.2	Hotel cohort by city	10
2.1	Crawl item outcome classes	14
2.2	Feature groups	16
2.3	Candidate models	17
2.4	Architecture layers	18
2.5	Technology stack	21
3.1	Milestones	27
3.2	Pilot results: 10 properties $\times$ 5 check-in dates $\times$ 6 crawl days	28
3.3	Risk assessment	29
4.1	Target values and their basis	32

List of Abbreviations

API	Application Programming Interface
ARIMA	AutoRegressive Integrated Moving Average
CSV	Comma-Separated Values
DOM	Document Object Model
DVC	Data Version Control
EDA	Exploratory Data Analysis
GSO	General Statistics Office of Vietnam
ICT	Indochina Time (UTC+7)
LSTM	Long Short-Term Memory
MAE	Mean Absolute Error
MAPE	Mean Absolute Percentage Error
ML	Machine Learning
MSE	Master of Software Engineering
OTA	Online Travel Agency
PMS	Property Management System
REST	Representational State Transfer
RMSE	Root Mean Squared Error
SHAP	SHapley Additive exPlanations
SQL	Structured Query Language
UI	User Interface
UTC	Coordinated Universal Time
VNAT	Vietnam National Administration of Tourism
VND	Vietnamese Dong
XGBoost	Extreme Gradient Boosting

Chapter 1

INTRODUCTION AND BACKGROUND

1.1 Problem Description

1.1.1 The Vietnamese Accommodation Market and OTA Distribution

Vietnam’s accommodation sector comprises more than 38,000 establishments, of which independent small and medium properties form the large majority [1]. Tourism volume has recovered past its pre-pandemic level, and the Vietnam National Administration of Tourism recorded approximately 17.5 million international arrivals in 2024 alongside a domestic travel market of comparable economic weight [1, 2]. Distribution for these properties runs largely through Online Travel Agencies. The 2023 e-Conomy SEA report estimated Vietnam’s online travel gross merchandise value at USD 5 billion, with accommodation as the largest component [3].

Two consequences follow for pricing. First, a property’s published rate is visible to every competitor in its area at all times, which makes rate setting a continuous competitive interaction rather than a seasonal planning exercise. Second, the OTA display itself changes within hours: partner rates appear and disappear, discount banners are applied and withdrawn, and the same room can carry different cancellation terms depending on the rate plan shown. A rate observed on Monday morning tells the observer very little about what the same room costs on Tuesday.

1.1.2 Dynamic Pricing and the Rate-Intelligence Gap

Hotels respond to this environment with dynamic pricing, adjusting rates according to remaining inventory, days to arrival, and observed competitor behaviour. Abrate et al. documented systematic rate variation by lead time across European hotels and showed that the direction of that variation depends on property characteristics rather than following a single industry rule [4]. Ivanov and Zhechev’s review of hotel revenue management describes competitor rate monitoring as a standard input to the pricing decision [5].

Chain hotels obtain that input from commercial rate-intelligence and revenue-management platforms such as RateGain, Lighthouse and Duetto, which are licensed at hundreds to thousands of US dollars per month [6, 7]. An independent 25-room property in Da Lat or

Vung Tau operates well below the revenue threshold at which such a licence is defensible. In practice these operators open competitor pages by hand, record a handful of rates in a spreadsheet, and set their own price from judgement. The resulting decision has three defects: it samples one moment rather than a trend, it covers a small and unstable set of competitors, and it carries no forward view of where the local rate level is heading.

Individual travellers face the same information problem from the opposite side. Comparison sites report the current rate across suppliers but do not retain a price history for a specific property, and consumer forecasting products such as Hopper cover Vietnamese inventory sparsely and do not model Vietnamese demand drivers [8].

1.1.3 Research Gap

Academic work on hotel pricing has concentrated on rate determinants and on aggregate demand forecasting. Ye et al. and Zhang et al. analysed how review volume and review score relate to room rates [9, 10], and Law et al. applied deep learning to tourism arrival forecasting for Hong Kong [11]. These studies model price or demand as a function of property and market attributes at a point in time. None of them models the trajectory of a specific room rate for a specific stay date as the arrival date approaches, which is the quantity a pricing or booking decision actually depends on.

Four gaps remain open for the Vietnamese market:

- **Longitudinal rate data with two time coordinates.** Published datasets record price snapshots. Forecasting how a rate moves requires repeated observation of the same stay date over successive days, which no public Vietnamese dataset provides.
- **Room comparability over time.** A rate series is only meaningful if consecutive observations describe the same product. OTA listings reorder room options between sessions and add partner rates asynchronously, so a naive rule such as “take the cheapest room” silently switches products and corrupts the series. Existing scraping studies rarely address this.
- **A decision layer for independent operators.** Research prototypes in this area target either the traveller or the revenue manager of a large chain. The competitive-set view that an independent hotel needs, showing its own position relative to a comparable group and the predicted movement of that group, has received little attention.
- **Vietnam-specific demand structure.** Lunar New Year, the June to August domestic peak, and city-level festivals such as the Da Lat Flower Festival shift demand in ways that Western-trained models do not encode. Representing these drivers requires a curated calendar keyed to both date and city.

This project addresses all four by building the collection pipeline, the comparability mechanism, the forecasting models, and the two decision layers as one system, and by evaluating the result on data the system itself collects.

1.2 Research Objectives

1.2.1 Main Objective

Design and build a web-based system that monitors Booking.com room rates for hotels in five Vietnamese cities, forecasts short-term rate movements at 1, 3, 7 and 14-day horizons using machine learning, and delivers those forecasts through a hotelier-facing competitive-set view and a consumer-facing booking-timing view served by a single forecasting model.

The architectural commitment behind the second half of that sentence is deliberate. The model performs one task: predict the rate of hotel h for check-in date d as observed k days from now. The consumer view reads that prediction for one hotel; the hotelier view aggregates it over a competitive set. Two application layers, one trained model.

1.2.2 SMART Goals

Table 1.1 states the objective against the SMART criteria, with each cell tied to a concrete deliverable or a measured threshold.

1.2.3 Research Questions

1. **RQ1.** How can a crawling pipeline collect Booking.com rates continuously over months while guaranteeing that consecutive observations of the same hotel and check-in date describe a comparable room and rate plan, and while distinguishing a sold-out night from an unbookable property and from a dead listing?
2. **RQ2.** Which of the four candidate model families (gradient boosting, bagged trees, recurrent networks, attention-based networks) forecasts short-term hotel rate movement most accurately at the 1, 3, 7 and 14-day horizons on this dataset, and how does accuracy vary with lead time, city and horizon?
3. **RQ3.** Which feature groups drive the forecast, measured by SHAP attribution and by ablation over the calendar, lead-time, price-history, competitive-set and external-context groups?
4. **RQ4.** To what extent do the forecasts support decisions? Measured for hoteliers as the number and magnitude of detected pricing-position deviations during predicted high-demand periods, and for consumers as the cost difference between booking on the system’s timing signal and booking immediately.

Table 1.1: SMART goal definition

Criterion	Description
Specific	Deliver three components: (1) a durable crawling pipeline that collects Booking.com rates for a fixed hotel cohort across multiple future check-in dates, with per-item completeness accounting; (2) four forecasting models (XGBoost, Random Forest, LSTM, Transformer) trained for four horizons; (3) a Next.js dashboard exposing competitive-set monitoring, market forecasts and pricing-position alerts for hoteliers, and price history with booking-timing guidance for consumers.
Measurable	$\text{MAPE} \leq 12\%$ at the 7-day horizon; price-direction accuracy $\geq 80\%$; $\text{RMSE} \leq 350,000$ VND; $R^2 \geq 0.75$; at least 100,000 price observations collected; crawl success rate $\geq 95\%$ per run; simulated booking savings $\geq 15\%$ against booking at first sight.
Achievable	The crawler, durable queue, reference-calibration logic and internal frontend are already implemented and validated on a six-day pilot (Section 3.3). The remaining work uses established libraries: scikit-learn, XGBoost, TensorFlow 2, Optuna and SHAP. No component requires model training from scratch or infrastructure beyond a single VPS.
Relevant	Independent hotels in Vietnam set prices without the competitor intelligence available to chains. The system provides that intelligence from public listing data at zero licence cost, and contributes a longitudinal Vietnamese hotel rate dataset that does not currently exist.
Time-bound	Twenty-four weeks from August 2026 to late January 2027, with data collection running continuously from week 3 so that the time series has depth before model training begins in week 9.

1.3 Scope of the Project

1.3.1 In Scope

- **Geography.** Ho Chi Minh City, Hanoi, Vung Tau, Da Lat and Phu Quoc.
- **Data source.** Booking.com only. Every observation is collected under a fixed query context: 2 adults, 0 children, 1 room, 1 night, currency forced to VND, with the checkout date always equal to the check-in date plus one.
- **Hotel cohort.** A master list of 355 properties distributed across the five cities (Table 1.2), rolled out in stages of 10, then 50, then the full list. Property tiering uses Booking.com review score and review count rather than official star rating, since the star field is not reliably present on the listing page.
- **Forecast horizons.** 1, 3, 7 and 14 days ahead of the observation date, for a fixed check-in date.
- **Models.** XGBoost, Random Forest, LSTM and Transformer, compared empirically. No model is designated in advance as the primary candidate.
- **External enrichment.** A Vietnamese holiday and event calendar built for this project (203 rows covering 2026 to 2028, keyed by date and city), weather forecasts from the Open-Meteo API [12], and monthly city-level tourist arrival statistics from VNAT and GSO publications.
- **Applications.** An internal scraper operations frontend (already implemented) and a product dashboard serving the hotelier and consumer views.
- **Evaluation.** Forecast accuracy metrics, SHAP interpretability, feature-group ablation, two retrospective decision simulations, and system-level measurements of crawl reliability, API latency, data freshness and retraining time.

Table 1.2: Hotel cohort by city

City	Properties in master list	Share
Hanoi	129	36.3%
Phu Quoc	63	17.7%
Da Lat	60	16.9%
Vung Tau	52	14.7%
Ho Chi Minh City	51	14.4%
Total	355	100%

1.3.2 Out of Scope

- Properties below the 3-star tier, homestays and short-term rental listings.
- Integration with a hotel Property Management System, channel manager, or any booking transaction.
- Native mobile applications. The dashboard is responsive for desktop and tablet.
- Flight or transport pricing.
- Usability testing with human participants. Evaluation is entirely quantitative, computed on held-out chronological data.
- Commercial deployment. The system is built for academic research on publicly visible listing data.

1.3.3 Deferred to Future Work

Agoda collection and every feature that depends on observing two OTAs at once were removed from the main scope. This covers rate parity checking, the cross-OTA price difference and ratio features, and the property-matching table that would link the same physical hotel across platforms.

The reason is operational. The crawler must run unattended for five to six months, and a gap of one week leaves a hole that breaks every lag and rolling feature computed near it. Maintaining one crawler that runs reliably protects the time series better than maintaining two, one of which is less mature. Adding Agoda later is a self-contained extension: it requires a new scraper module and a property-matching migration, and it does not invalidate the Booking.com dataset or the model architecture.

Chapter 2

PROPOSED SOLUTION

2.1 Solution Description

The system has four subsystems: a durable collection pipeline, a room-comparability mechanism, a feature and labelling pipeline, and a forecasting engine feeding two application views. This section describes each, beginning with the data model that constrains all of them.

2.1.1 The Two Time Coordinates

Every price observation carries two dates that vary independently:

- `observed_at`, the moment the crawler read the page, stored in UTC;
- `checkin_date`, the night of stay the rate applies to.

The rate for 20 December seen today differs from the rate for 20 December seen next week. Collapsing these into one date, which a single-snapshot crawl does implicitly, destroys the only signal a booking-timing or repricing decision can use. From the pair the pipeline derives

$$\text{lead_time} = \text{checkin_date} - \text{DATE}(\text{observed_at}),$$

which is computed at insert time and stored on the row, so no later stage recomputes it.

Each crawl run therefore visits several future check-in dates per hotel. After N crawl days across M check-in dates, the dataset holds $N \times M$ points per hotel, arranged as a grid of observation times across stay dates. Figure 2.1 shows the structure: a column is one stay date observed repeatedly, and the diagonal of each column traces the rate as lead time shrinks.

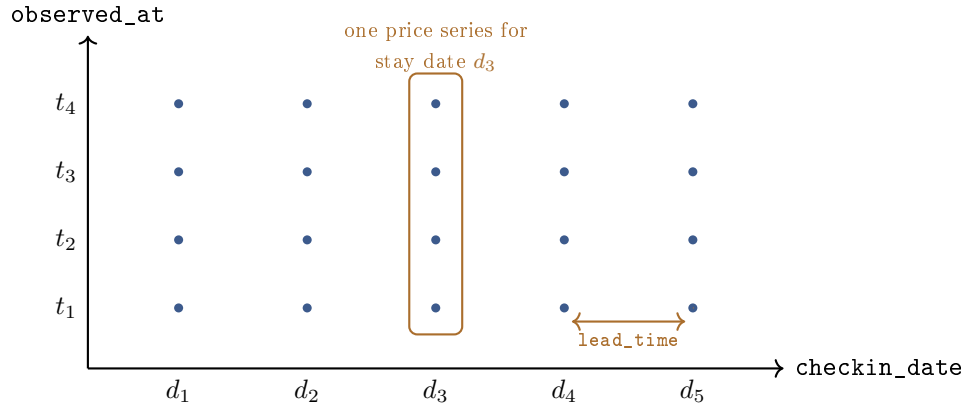


Figure 2.1: Observation grid. Each dot is one crawl of one hotel for one stay date. A column is a single rate series observed at successive times; the horizontal distance from an observation to its stay date is the lead time.

2.1.2 Data Collection Subsystem

Booking.com renders room tables in JavaScript and inserts partner rates asynchronously, so collection uses Selenium with a real Chrome instance rather than an HTTP client. The crawler waits at least eight seconds after the first room table appears and reads only once the number of rate rows has stabilised, which prevents recording a partial table as a complete one. The driver runs headless at a 1920×1080 desktop viewport and reads values from DOM `textContent` instead of Selenium’s `.text` property, because Booking’s CSS returns empty strings for visible text at narrow headless viewports.

Execution is split between two independent processes that share MySQL as a durable queue. The API validates an uploaded Excel workbook, creates one `crawl_run` row and one `crawl_run_item` row per (hotel, check-in date) pair, and returns immediately. A separate worker process claims items one at a time under a lease, writes a heartbeat while working, and releases or reclaims the lease on completion or expiry. Closing the browser or restarting the API does not affect a running job, and a worker crash does not leave items permanently claimed. The worker reuses one Chrome session for ten consecutive items before restarting it, which bounds memory growth without paying driver startup cost on every request.

Progress is written to the database continuously and read by the frontend through a polling endpoint. The project uses no WebSocket, no Celery and no Redis; Section 2.3.1 explains why.

Four outcome classes are kept distinct because conflating them corrupts the dataset in different ways:

A sold-out night is a demand signal and belongs in the dataset. An unbookable property is a lifecycle state of the property and belongs on the `hotels` row. A dead link is a collection failure. Writing any of these as a zero price would inject three different

Table 2.1: Crawl item outcome classes

Status	Meaning	Recorded as
success	Room table parsed and every parsed option persisted	One observation per rate option
partial	Parsed and persisted counts disagree	Observations kept, item flagged for audit
sold_out	No inventory for that night at that property	One observation with <code>price = NULL</code> and <code>availability_status = sold_out</code> , never price 0
not_bookable	Property-wide banner stating the hotel cannot be booked	No price observation; <code>hotels.booking_status</code> updated and the remaining queued dates for the same link are circuit-broken
error	Dead link, redirect to search results, timeout, driver failure, CAPTCHA or block	Classified by error code, retryable or permanent

fiction into the price series.

Every item records the counts that make completeness auditable: DOM room rows found, candidate rates, options parsed, options rejected, and options written to the database. When the parsed count and the saved count disagree, the item is marked `partial` and excluded from the success count. This check exists because an earlier design used a deduplication key derived from the normalised room-type name, which silently discarded 30 of 47 valid rate options in a four-hotel test. The current unique key is `(crawl_run_item_id, room_option_index)`, and inserts are strict, with no `INSERT IGNORE`.

Each item also stores its requested and final URL, an error taxonomy code, retry and lease timestamps, per-stage timings, and the scraper, selector and git versions in use, so that a later anomaly in the data can be traced to the code that produced it.

2.1.3 Reference Room Calibration

A price series is comparable over time only if consecutive observations describe the same product. Booking.com reorders rate options between sessions, and partner rates appear and disappear, so neither DOM position nor “cheapest available” identifies a stable product. Creating a reference room by hand for hundreds of properties is not workable either.

The system derives two hashes from attributes that do not depend on price or DOM order:

- `room_identity_key`: SHA-256 over the canonicalised room name, maximum occupancy, bed configuration and normalised area. Canonicalisation strips Vietnamese dia-

critics, lowercases, and maps known synonyms (`giuong doi` to `double`, `nhin ra bien` to `sea view`).

- **rate_plan_key**: SHA-256 over breakfast inclusion, free-cancellation flag and canonicalised cancellation policy.

Candidates accumulate in `hotel_room_candidates`, scoped to **(hotel_id, checkin_date)** rather than to the hotel alone. Each stay date is a separate price series with its own inventory, so a single hotel-wide reference would compare rates across nights whose room availability differs. A candidate is promoted to an approved reference only when all three conditions hold:

1. it appears in at least 3 completed crawl runs;
2. its coverage across the distinct raw-complete items of that same (hotel, check-in date) series reaches at least 80%;
3. exactly one option matches it within each item, so the reference is unambiguous.

Before approval a candidate stays in **proposed** or **calibrating** state and is excluded from the training set. Matching a later observation to an approved reference yields **exact** when both hashes agree, or **alias** when the rate plan agrees and a weighted similarity over name tokens, occupancy and area reaches 0.80. Two equally scored matches yield **ambiguous** and no reference assignment.

Reference availability and crawl completeness are tracked as separate indicators. An item whose room table parsed and persisted completely is a **success** even when its reference is unavailable or ambiguous for that night, because the raw observations remain valid data.

2.1.4 Feature Engineering and Label Generation

Features fall into five groups, all derived from collected data without additional requests:

Lag and rolling features hold **checkin_date** fixed and shift **observed_at**. A lag computed across different stay dates would mix unrelated series and carries no meaning. Weather at prediction time must come from a forecast, since realised weather is unavailable at the moment the prediction is made.

Labels come from the series itself and require no manual annotation. For an observation at (h, d, t) and horizon $k \in \{1, 3, 7, 14\}$, the pipeline locates the observation with the same hotel, the same check-in date, and **observed_at** = $t + k$, then computes

Table 2.2: Feature groups

Group	Features
Calendar	Day of week, weekend flag, month, week of year, quarter, season, public holiday flag, holiday-eve flag, days to nearest holiday, Tet period flag, city-level festival flag
Lead time	<code>lead_time</code> , bucketed lead time (<3, 3–7, 7–14, 14–30, 30–60, >60 days), last-minute flag
Price history	Lags at $t-1$, $t-3$, $t-7$, $t-14$; rolling mean over 7 and 14 days; rolling standard deviation over 7 days; price velocity; trailing 14-day minimum and maximum; ratio to the property’s own 30-day mean; days since last price change
Competitive set	Competitive-set mean, median, minimum and maximum price for the same check-in date and observation time; own price to set mean ratio; price rank within set; set mean lagged 7 days; set price velocity; share of the set sold out as a demand proxy
External context	Temperature, precipitation probability, humidity from Open-Meteo; monthly city tourist arrivals; major event flag

$$y_{\text{price}}^{(k)} = p_{t+k}, \quad y_{\Delta}^{(k)} = p_{t+k} - p_t, \quad y_{\%}^{(k)} = \frac{p_{t+k} - p_t}{p_t},$$

$$y_{\text{dir}}^{(k)} = \begin{cases} \text{up} & y_{\%}^{(k)} > +0.02 \\ \text{down} & y_{\%}^{(k)} < -0.02 \\ \text{stable} & \text{otherwise.} \end{cases}$$

The $\pm 2\%$ band defining “stable” will be revisited after exploratory analysis of the observed change distribution. When no observation exists at $t+k$, the row sets `has_label_hk` = `false` and drops out of that horizon’s training set while remaining usable for others. Sold-out rows carry no price label; predicting sold-out probability is a possible secondary task and is not mixed into the price regression.

Each row also stores `split`, `is_anomaly`, `is_complete`, `label_source_record_id` and `data_version` so that any training row can be traced to the raw observations behind it.

2.1.5 Forecasting Engine and Two Decision Layers

Four models are trained and compared. Each represents a distinct methodological family, so the comparison contrasts approaches instead of near-duplicates:

LightGBM and CatBoost are excluded because they differ from XGBoost mainly in engineering optimisation, and GRU is excluded for the same reason relative to LSTM.

Table 2.3: Candidate models

Model	Family	Rationale
XGBoost [13]	Gradient boosting	Sequential error correction; boosted trees remain the strongest general approach on tabular data with heterogeneous feature types [14]
Random Forest [15]	Bagging	An independent ensemble principle, reducing variance where boosting reduces bias; stable under minimal tuning and a useful reference point for the boosting result
LSTM [16]	Recurrent network	Gated memory over the observation sequence of a fixed stay date; established in tourism demand forecasting [11]
Transformer [17]	Attention	Attention over the observation window instead of recurrence, giving a recurrent-versus-attention contrast whose outcome on short series is contested [18]

ARIMA and exponential smoothing are excluded because they cannot use the exogenous features that carry most of the signal here: holidays, weather, competitive-set position and lead time.

The trained model is served to two views:

Hotelier view. A property selects a competitive set of 5 to 15 comparable hotels, defined by city, proximity in review score and a minimum review count, with address tokens available as a tie-breaker. The dashboard shows current competitor rates, the forecast rate level of the set over the next 14 days, and an alert when the property’s own rate deviates materially from the set or when the model predicts a market movement around a holiday or event.

Consumer view. For one hotel and one stay date, the dashboard shows the observed rate history and the forecast, and converts the forecast into booking-timing guidance.

2.2 Software Architecture

2.2.1 Component Layers

Table 2.4 lists the layers and what each one owns. Figure 2.2 shows how they connect.

Table 2.4: Architecture layers

Layer	Technology	Responsibility
Presentation	Next.js (two applications)	scraper-frontend for internal operations: workbook upload, check-in date selection, run progress, item-level diagnostics, artifact download, Excel export. dashboard-frontend for the hotelier and consumer views.
Application	FastAPI (Python)	One service exposing <code>/api/scraper/*</code> for operations and <code>/api/dashboard/*</code> for the product views, plus model serving.
Job execution	Python worker over a MySQL queue	Lease-based item claiming, heartbeat, retry with error classification, stale-lease reclamation, driver lifecycle.
Collection	Selenium 4, BeautifulSoup	Page rendering, availability wait, DOM extraction, optional HTML and screenshot capture.
Data	MySQL 8	Hotels, price observations, run and item queue, room candidates and references, holidays, weather, tourism statistics, cleanup audit.
ML	scikit-learn, XGBoost, TensorFlow 2, Optuna, SHAP	Feature construction, training, tuning, interpretability.
Packaging	Docker, Docker Compose	Reproducible environments for backend, worker and both frontends.

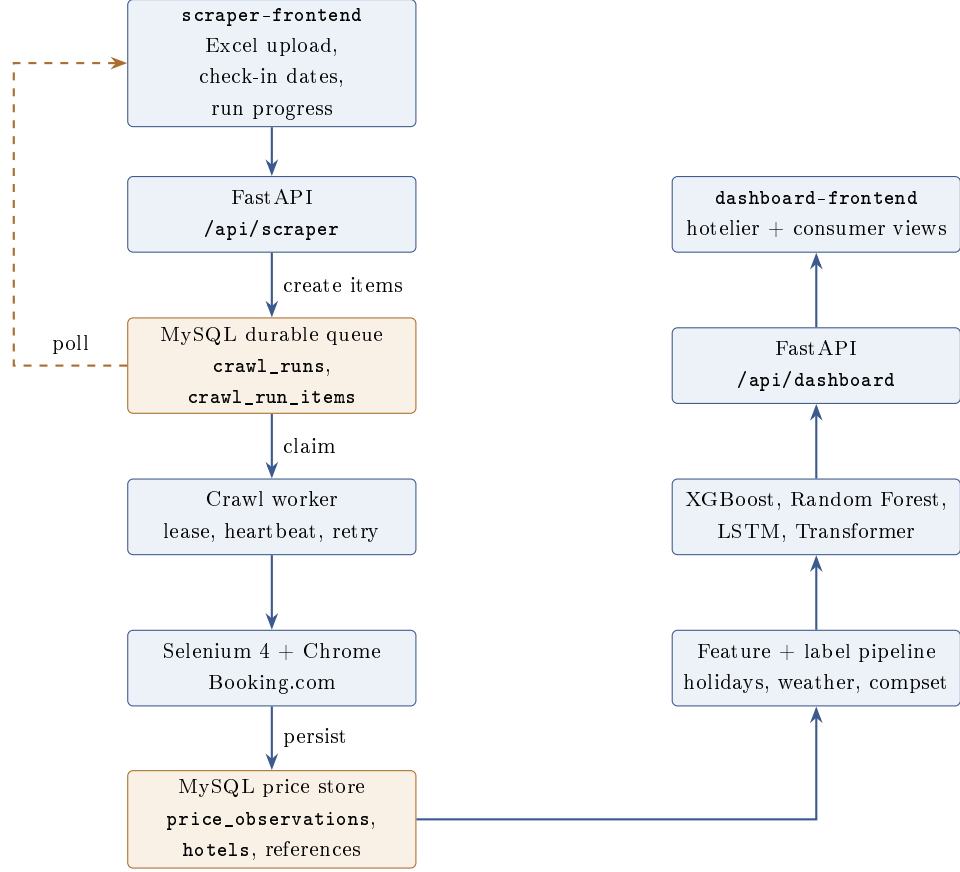


Figure 2.2: System architecture. The API and the crawl worker are separate processes coordinating only through the MySQL queue, so neither depends on the other staying alive.

2.2.2 Database Schema

The schema separates near-static property attributes from the observation stream. **hotels** holds one row per property keyed by the Booking.com URL slug, which is stable across crawls and serves as the internal **hotel_id**. Since the scope covers one OTA, no property-mapping table is needed.

price_observations is the large table, holding one row per rate option per crawl of one hotel and stay date. It carries the two time coordinates, the precomputed lead time, price and discount fields, the tax fields (**taxes_fees** stays NULL unless the page states a separate amount, and **price_includes_tax** is parsed from the source sentence rather than inferred), room and rate-plan attributes, both identity hashes, the reference match status and score, the rooms-left counter, availability status and an anomaly flag. Indexes cover (**hotel_id**, **checkin_date**, **observed_at**) and (**checkin_date**, **observed_at**), with the unique constraint on (**crawl_run_item_id**, **room_option_index**).

Supporting tables cover the queue (**crawl_runs**, **crawl_run_items**, **crawler_workers**), room comparability (**hotel_room_candidates**, **hotel_reference_rooms**), retention au-

dit (`file_cleanup_logs`), and enrichment (`vn_holidays`, `weather_data`, `tourism_stats`, `competitive_sets`). Tables for `ml_features`, `predictions` and `alerts` are added in Phases 3 and 6.

The holiday table deserves a note because no suitable source existed to import; the calendar was built for this project. It holds 203 rows spanning 2026 to 2028: 132 confirmed and 71 provisional, 36 national and 167 city-scoped, split as 33 public holidays, 147 festival days and 23 major events. Every day of a multi-day event is one row, keyed by (`holiday_date`, `event_code`) so that overlapping events on the same date do not collide. Lunar-calendar dates are converted to the Gregorian calendar and marked provisional until the responsible authority publishes the official schedule. A CHECK constraint enforces that national rows have a NULL city and city rows do not, so the table joins directly to `hotels.city`.

2.2.3 Data Flow

1. The operator uploads an Excel workbook with one sheet per city and selects one or more check-in dates. Preflight validates link syntax, duplicates and sheet scope before submission.
2. The API creates the run and one queued item per (hotel, check-in date) pair, then returns the run identifier. Checkout is always check-in plus one day.
3. The worker claims an item under a lease, builds the canonical Booking.com URL with the fixed query context, loads the page, waits for the room table to stabilise, and extracts the room options.
4. The worker classifies the outcome, upserts the hotel row, builds the observation rows, and commits observations and item status in one transaction. Nothing is written for an item that failed to parse completely.
5. On run completion the system refreshes room candidates and promotes eligible ones to approved references.
6. Phase 3 onward: the feature pipeline joins observations with holidays, weather and tourism statistics, computes the five feature groups, generates labels for the four horizons, and exports a flat parquet dataset for the tree models and a windowed tensor dataset for the sequence models.
7. Trained models produce forecasts served through `/api/dashboard` to the hotelier and consumer views.

Artifact capture is opt-in and off by default. When enabled before a run, the worker stores the gzipped HTML and a screenshot taken at the moment of the crawl. Uploads

are retained for 90 days and artifacts for 30 days, with cleanup triggered at worker startup or run manually. Artifacts serve audit only and never enter the training set.

2.3 Technology and Tools Used

Table 2.5: Technology stack

Category	Technology	Rationale
Frontend	Next.js	React-based, so it satisfies the React dashboard commitment in the approved topic description while providing routing and server rendering without extra configuration. Two applications share conventions and call one backend.
Backend	FastAPI	Async request handling, Pydantic request validation, generated OpenAPI documentation [19].
Database	MySQL 8	Relational structure fits the hotel-to-observation hierarchy; the durable queue relies on row-level locking and transactional commit of observations together with item status.
Job execution	Python worker over MySQL	Fewer moving parts than a broker-based queue for a single-worker sequential workload; job state survives process restarts because it lives in the database.
Collection	Selenium 4, BeautifulSoup	Booking.com renders rates in JavaScript and inserts partner rates asynchronously, so a rendering browser is required. Selenium CDP also allows geolocation and timezone emulation for Vietnam.
ML	scikit-learn, XGBoost, TensorFlow 2	Covers the tree-based and deep-learning families in the comparison.
Tuning	Optuna	Bayesian search over hyperparameters with pruning [20].

Category	Technology	Rationale
Interpretability	SHAP	Additive feature attribution, giving both global feature ranking and per-prediction explanation [21].
Hosting	VPS (projecthub.io.vn)	Runs the crawler continuously and hosts the backend and frontends.
Packaging	Docker, Docker Compose	Reproducible environments across development and deployment.

2.3.1 Deviations from the Approved Topic Description

The topic description submitted for approval named PostgreSQL with Redis, Celery for scheduling, Scrapy alongside Selenium, and collection from both Agoda and Booking.com. Implementation departed from each of these. The reasons are recorded here so that the thesis can state them once and move on.

- **Booking.com only.** See Section 1.3.3. The decision protects continuity of the time series, which is the binding constraint on the whole project.
- **MySQL instead of PostgreSQL, and no Redis.** The schema uses no engine-specific feature. MySQL was already provisioned and understood on the target VPS, and one fewer service to keep alive over five months of unattended operation reduces the failure surface. Redis was introduced in the original plan as a cache and a Celery broker; with Celery removed and query volume from a single-operator dashboard low, it has no remaining function.
- **Durable MySQL queue instead of Celery.** Celery and Redis add two processes whose failure modes are independent of the crawl itself. Placing the queue in the same transactional store as the results means an item’s status and its observations commit or roll back together, which removes the class of bug where a job is marked complete but its data is missing.
- **No scheduler.** Runs are created by the operator, who chooses the check-in dates explicitly before starting. Automatic scheduling was deliberately dropped: with a single crawler on a single IP address, an unattended weekly job that starts failing produces a silent gap in the series, whereas an operator-initiated run is inspected when it finishes. A written sampling protocol keeps the collection cadence regular.
- **No WebSocket.** Progress lives in the database. The frontend polls every 20 to 30 seconds while a job page is open and stops when the job ends. Closing the browser does not affect the job.

- **Selenium only, no Scrapy.** Scrapy adds value for large-scale structured crawling of static pages. Every page here requires JavaScript rendering, so Scrapy would sit in front of Selenium contributing scheduling machinery the durable queue already provides.
- **Next.js instead of plain React.** Next.js is a React framework, so this choice sits inside the approved commitment.

None of these changes affect the research methodology, the model comparison, or the evaluation design.

Chapter 3

IMPLEMENTATION PLAN

3.1 Main Development Stages

The project runs for 24 weeks from August 2026 to late January 2027 across eight phases. Data collection is the binding constraint: it is limited by calendar time rather than by effort, because eight to twelve weeks of uninterrupted observation are needed before lag and rolling features have enough history to be computed. Collection therefore starts in week 3 and continues in parallel with every later phase. Phases 3 through 6 overlap by design.

3.1.1 Phase 0 – Foundation (Weeks 1–2)

- Assemble the hotel master list across the five cities and store it as an Excel workbook with one sheet per city.
- Design and create the MySQL schema described in Section 2.2.2.
- Port the Booking.com collection logic from the earlier prototype, retaining the anti-detection driver configuration, the CSS selector fallback chains and the currency and date forcing in the URL.
- Build the Vietnamese holiday and event calendar, converting lunar dates and marking unpublished future schedules as provisional.

3.1.2 Phase 1 – Collection Application and Durable Worker (Weeks 2–3)

- Implement the MySQL durable queue with lease-based claiming, heartbeat, classified retry and stale-lease reclamation.
- Separate the worker process from the API so that job execution survives an API restart.
- Build the internal frontend: workbook upload with preflight validation, explicit check-in date selection, run progress, item-level diagnostics with candidate, parsed and saved counts, failed-item retry, artifact download and Excel export.

- Implement automatic reference calibration per (hotel, check-in date).

3.1.3 Phase 2 – Continuous Collection (Week 3 onward, in parallel)

- Run a fixed pilot cohort to validate the pipeline end to end before scaling.
- Expand in stages: 10 properties, then 50, then the full master list. Each stage is audited before the next begins.
- Track weekly: crawl success rate, accumulated observation count, partial and rejected option rates, missingness by field, reference coverage and approval rate, and whether Booking.com has changed its DOM structure.
- Refresh property attributes monthly and back up the database on a fixed schedule.

3.1.4 Phase 3 – Exploratory Analysis, Features and Labels (Weeks 5–9)

- Analyse price distributions, sold-out rates, missingness and anomalous prices per property.
- Integrate Open-Meteo weather and VNAT tourism statistics by city and date.
- Define competitive sets from city, review score proximity and minimum review count, and persist them.
- Implement the feature pipeline as reusable Python modules covering all five feature groups, with notebooks reserved for exploration.
- Implement label generation for the four horizons, including anomaly flags and per-horizon label availability.
- Split chronologically 70/15/15 with a gap of 14 days, the longest horizon, between adjacent splits so that a training row's label cannot reach into the validation period.
- Export the flat parquet dataset and the windowed tensor dataset, fitting scalers on the training split only.

3.1.5 Phase 4 – Model Development (Weeks 9–16)

- Train XGBoost and Random Forest first on the tabular dataset for all four horizons, establishing a reference result quickly.

- Train LSTM and Transformer on the windowed dataset.
- Tune all four with Optuna, fix random seeds, and record hyperparameters in `ml/configs/*.yaml`, never hard-coded.
- Decide and document whether each horizon gets its own model or one model emits multi-horizon output.

3.1.6 Phase 5 – Evaluation and Interpretability (Weeks 16–19)

- Compute MAPE, RMSE, MAE and R^2 per model and horizon, plus direction accuracy and precision and recall for the price-drop class.
- Run SHAP on the best model, reporting the top ten features overall and broken down by city and review-score tier.
- Run the ablation study over the five feature groups.
- Select the best model on measured results.

3.1.7 Phase 6 – Dashboard and Decision Simulations (Weeks 17–22)

- Implement prediction serving: load the trained model and expose forecasts by hotel, check-in date and horizon.
- Build the hotelier view: competitive-set dashboard, market forecast panel, pricing-position alerts.
- Build the consumer view: price history with forecast overlay and booking-timing guidance.
- Run the pricing opportunity analysis and the booking savings simulation on the held-out period.

3.1.8 Phase 7 – Thesis and Defence (Weeks 20–24)

- Write the methodology, data, experiment, results, discussion and limitations chapters.
- Record the implementation decisions of Section 2.3.1 in the implementation chapter.
- Measure system performance: API p95 latency, data freshness, retraining time.
- Prepare the defence demonstration and reserve the final two weeks for supervisor revisions.

3.2 Expected Schedule

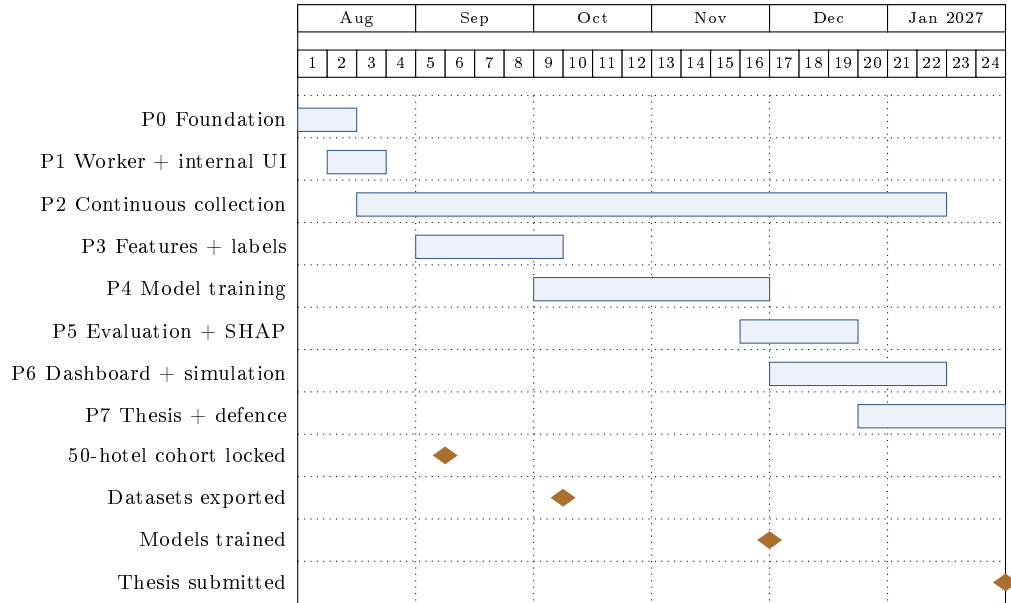


Figure 3.1: Project timeline over 24 weeks, August 2026 to late January 2027. Collection runs continuously alongside every phase after week 3.

Table 3.1: Milestones

Week	Milestone
3	Durable worker operational; pilot cohort passing the data-quality gate.
5	50-property cohort locked with a versioned workbook and collection under way.
9	Full cohort in collection; table and sequence datasets exported with labels for all four horizons.
16	Four models trained and tuned across four horizons, reproducible from configuration files.
19	Comparative results, SHAP analysis and ablation study complete; best model selected.
22	Dashboard operational for both views; both decision simulations run on the held-out period.
24	Thesis submitted; defence prepared.

3.3 Current Implementation Status

Phases 0 and 1 are complete and Phase 2 is under way. The figures below come from runs already executed against Booking.com and audited with the project’s own scripts; they are reported here as evidence of feasibility rather than as research results.

Pipeline. The collection application, durable worker, reference calibration and internal frontend run locally. The backend test suite contains 15 tests, all passing. The crawler is at version 2.2.0 with selector set `booking-2026-08-12`.

Pilot cohort. A fixed cohort of 10 properties, two per city, was crawled for 5 check-in dates on 6 separate calendar days, giving 300 crawl items. Table 3.2 reports the outcome. No item finished in `partial` or `error` state, no CAPTCHA or block was encountered, and no duplicate violated the `(crawl_run_item_id, room_option_index)` constraint. For every item that returned a price, the candidate, parsed and saved option counts matched.

Table 3.2: Pilot results: 10 properties $\times$ 5 check-in dates $\times$ 6 crawl days

Measure	Value
Crawl items attempted	300
Items completed successfully	250
Items sold out for the requested night	20
Items on properties flagged not bookable	30
Items partial or in technical error	0
Price observations stored	2,312
Duplicate observations	0
CAPTCHA or block events	0

Reference calibration. Of the 42 (hotel, check-in date) series that had inventory during the pilot, 38 reached approved reference status and 4 remained proposed, a readiness of 90.5% against the 80% gate. No observation was linked to a reference belonging to a different check-in date.

Artifact-level validation. A separate regression run covering 2 cities, 1 property each and 2 check-in dates stored 51 observations across 4 items. A validator re-read the gzipped HTML captured at the moment of each crawl and compared 51 DOM rows against 51 database rows across 473 fields, finding no mismatch. Property identity, final URL, check-in and checkout dates, occupancy context, night count and currency all matched.

Reopening the same Booking.com pages after the crawl returned a different number of partner rates for one property, which confirms that inventory shifts between sessions. The audit therefore treats the artifact captured during the crawl as the authoritative record and uses a later browser session only for sanity checking, never as an oracle for past inventory.

Defects found and fixed. Three problems surfaced during validation and were corrected before scaling. A deduplication key based on the normalised room-type name discarded 30 of 47 valid rate options in an early test; the key is now `(crawl_run_item_id, room_option_index)` with strict inserts. Reading text through Selenium’s `.text` property returned empty strings at a narrow headless viewport, causing one Phu Quoc property

to be recorded as having no rooms while a 550,000 VND rate was present in the DOM; extraction now uses `textContent` at a 1920×1080 viewport. The circuit breaker for unbookable properties copied the first date’s final URL onto its sibling items; the code was fixed and the 8 affected historical rows repaired so that every item carries the URL for its own check-in date.

Next step. The 50-property cohort, ten per city, is prepared and awaiting its first batch run.

3.4 Feasibility Assessment

3.4.1 Risks and Mitigations

Table 3.3: Risk assessment

Risk	Severity	Mitigation
Booking.com changes its DOM and breaks selectors mid-collection	High	Selector chains with fallbacks and a versioned selector set; per-item candidate, parsed and saved counts make a silent extraction failure visible as a count mismatch instead of a quietly empty result; DOM checks repeated weekly, not only at project start.
Gap in the time series from crawler downtime	High	Job state lives in MySQL, so an API or browser restart does not lose a run; stale leases are reclaimed automatically; database backups on a fixed schedule; a gap of more than a few days triggers an investigation before more data is collected on top of it.
IP rate limiting or blocking	Medium	Randomised delays between requests, one worker per IP address, no parallel runs, and volume kept to operator-initiated batches; CAPTCHA and block events are recorded as their own error codes so that the rate is measurable.

Risk	Severity	Mitigation
Insufficient data volume by the training deadline	Medium	Collection starts in week 3 rather than after the ML pipeline is built; the cohort scales in stages so that each stage adds volume while the previous one keeps running; the master list of 355 properties across 5 check-in dates yields roughly 1,775 items per crawl day, and the pilot averaged 9.2 observations per successful item.
Room comparability drift over months	Medium	Reference definitions are scoped per (hotel, check-in date) and require three runs plus 80% item coverage before approval; unapproved candidates are excluded from training; ambiguous matches are recorded and left unassigned.
Model overfitting or leakage	Medium	Chronological split with a 14-day gap between splits; scalers fitted on training data only; lag features computed within a fixed check-in date; forecast weather only.
Reproducibility	Low	Fixed random seeds, hyperparameters in configuration files, Docker for environment consistency, and parser, selector and git versions recorded on every crawl item.

3.4.2 Resource Requirements

Development runs on a personal machine with 16 GB RAM and GPU support for the sequence models. A VPS already in use for other work hosts the crawler, MySQL and the deployed services. Every library in Table 2.5 is open source. All data comes from publicly visible Booking.com listings and free public sources: Open-Meteo, VNAT and GSO publications, and the holiday calendar built for this project. No licensed dataset and no paid proxy service is required.

3.4.3 Ethical Considerations

Collection is limited to publicly visible listing pages. The crawler accesses no user account and no personal data, uses randomised delays between requests, respects `robots.txt`, and keeps request volume to operator-initiated batches, with no continuous high-frequency polling. Stored data covers property attributes and room rates only. The dataset is used for academic research and is not redistributed commercially.

Chapter 4

EXPECTED RESULTS

4.1 Deliverables

1. **A longitudinal Vietnamese hotel rate dataset.** At least 100,000 price observations covering properties in five cities, each row carrying both an observation timestamp and a stay date, with room and rate-plan identity hashes, reference match status, availability status and provenance fields. The pilot averaged 9.2 stored observations per successful item at a 92.6% success rate once the one unbookable property is removed from the cohort, so a full-cohort crawl day across 355 properties and 5 check-in dates produces on the order of 15,000 observations. On that basis the 100,000 target is reached within roughly seven full-cohort crawl days, and collection continues well past it to lengthen the series rather than only to enlarge it. The dataset ships with a data dictionary and a data-quality report covering missingness, anomaly rate and reference coverage per city.
2. **Four trained and compared models.** XGBoost, Random Forest, LSTM and Transformer, each trained for the 1, 3, 7 and 14-day horizons, with hyperparameter configurations, training curves and held-out test metrics. Every result is reproducible from a configuration file and a fixed seed.
3. **A comparative analysis report.** Accuracy per model and horizon, breakdown by city, review-score tier and lead-time bucket, SHAP feature attribution for the selected model, and an ablation study measuring the contribution of the calendar, lead-time, price-history, competitive-set and external-context groups.
4. **A working system.** The internal operations application (already implemented) plus the product dashboard: a FastAPI backend serving both route groups, a Python crawl worker over the MySQL durable queue, two Next.js applications, and a Docker Compose configuration for deployment.
5. **Two decision simulations.** A pricing opportunity analysis quantifying detected mispricing for the hotelier view, and a booking savings simulation quantifying the cost difference from following the timing signal for the consumer view. Both run retrospectively on the held-out period.

6. The thesis and defence materials.

4.2 Performance Targets

Table 4.1 lists the target for each metric together with the reasoning that fixes it at that value.

Table 4.1: Target values and their basis

Metric	Target	Basis
MAPE, 7-day horizon	$\leq 12\%$	Within the range reported for comparable hotel price and tourism demand forecasting work [11, 10]. Primary metric because it is scale-invariant across properties whose rates differ by an order of magnitude.
RMSE, 7-day horizon	$\leq 350,000$ VND	Roughly 10% of a typical nightly rate in the 3 to 5-star segment covered here.
R^2	≥ 0.75	The model accounts for at least three quarters of rate variance on held-out data.
Price direction accuracy	$\geq 80\%$	Direction is the signal both views act on. Four correct calls in five is the threshold at which acting on the signal beats ignoring it.
Simulated booking savings	$\geq 15\%$	Measured against booking at the first observed price on the same stay date.
Crawl success rate	$\geq 95\%$	Per run, counting technical failures only. Sold-out nights and unbookable properties are valid outcomes, not failures.
API latency, p95	≤ 200 ms	Dashboard responsiveness under simulated concurrent load.
Data freshness	≤ 1 hour	Time between a crawl completing and its observations being visible in the dashboard.
Retraining time	≤ 30 minutes	Full retraining on the accumulated dataset.

The accuracy targets are hypotheses to be tested, not claims. If a model family falls short, the result is reported as measured and the gap analysed by horizon, lead-time bucket and city, since a model that forecasts well at 14 days and poorly at 1 day, or well in Hanoi and poorly in Phu Quoc, is a more useful finding than a single averaged number.

4.3 Contributions

- A longitudinal Vietnamese hotel rate dataset with two independent time coordinates, which no public source currently provides.
- A room-comparability mechanism that keeps a rate series describing one product over months without manual per-property configuration, together with the evidence that a naive deduplication rule fails at this task.
- A comparison of four model families on short-horizon hotel rate forecasting for a South-east Asian market, with feature-group attribution.
- A rate-intelligence design aimed at independent operators rather than at chains, evaluated by simulation on held-out data rather than by feature listing.

Chapter 5

EVALUATION PLAN

Evaluation is entirely quantitative and runs on the chronologically final 15% of the collected data, which no model sees during training or tuning. It covers four areas: forecast accuracy, interpretability, decision value, and the reliability of the data and the system that produced it. No usability study with human participants is conducted.

5.1 Forecast Accuracy

5.1.1 Regression Metrics

All four models are evaluated at all four horizons on the held-out test period.

Mean Absolute Percentage Error is the primary metric because rates in the cohort span an order of magnitude and a scale-dependent error would be dominated by expensive properties:

$$\text{MAPE} = \frac{1}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| \times 100\%$$

Root Mean Squared Error penalises large errors more heavily and exposes worst-case behaviour that MAPE averages away:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Mean Absolute Error gives the typical error in VND, which is the unit a hotelier reasons in. R^2 reports the share of rate variance the model accounts for.

Results are reported in aggregate and broken down by horizon, lead-time bucket, city and review-score tier. A model whose overall MAPE meets the target while failing inside the last-minute bucket is not usable for the decision the system supports, and an aggregate figure would hide that.

5.1.2 Direction Classification

Both views act on the direction of movement rather than on the exact rate, so direction is evaluated as a separate three-class problem over up, down and stable as defined in Section 2.1.4:

- Overall direction accuracy across the three classes.
- Precision and recall for the price-drop class specifically. Recommending “wait” when the rate is about to rise is the expensive error for a consumer, and predicting a market softening that does not occur is the expensive error for a hotelier, so both sides of this class matter.
- A confusion matrix per horizon, since the stable band absorbs a growing share of cases as the horizon shortens.

5.1.3 Baselines

Model results are compared against two naive references so that the reported accuracy has a floor to beat: a persistence baseline that predicts the last observed price for the same stay date, and a seasonal baseline that predicts the price observed on the same weekday of the previous week for the same stay date. A model that does not beat persistence at a given horizon is reported as such.

5.2 Interpretability

SHAP analysis [21] is applied to the selected model to answer RQ3:

- Global ranking of the ten most influential features, with the expectation that lead time, own-price lags and competitive-set position dominate, and with the actual ranking reported whatever it turns out to be.
- Dependence plots showing the direction and shape of each top feature’s effect, for example how the predicted rate responds as lead time falls from 21 days to 3.
- Breakdown by city and review-score tier, which tests whether the drivers differ between a Hanoi city-centre property and a Phu Quoc resort.

An **ablation study** complements the attribution by retraining the selected model with one feature group removed at a time, over the calendar, lead-time, price-history, competitive-set and external-context groups. SHAP measures attribution within a fitted model; ablation measures what the group is worth to the model overall. The competitive-set group is the one this project adds beyond standard price-history forecasting, so its ablated contribution is reported as a named result.

5.3 Decision Value

5.3.1 Pricing Opportunity Analysis (Hotelier View)

Run retrospectively over the test period:

1. For each property and each observation date, compute the forecast movement of its competitive set over the next 7 days.
2. Identify cases where the property's own rate moved against the predicted set movement, or stayed flat while the set was predicted to rise, during periods flagged as high demand by the holiday and event calendar.
3. For each such case, quantify the gap in VND between the property's rate and the realised set level, and record how many days ahead of the realised movement the alert would have fired.
4. Report the count of detected cases, the distribution of gap magnitude, and the distribution of alert lead time, broken down by city.

The output is a measure of how often and how early the system surfaces a pricing position worth reviewing. It does not claim that acting on every alert would have increased revenue, since the dataset contains rates and not bookings.

5.3.2 Booking Savings Simulation (Consumer View)

1. For each (hotel, check-in date) series in the test period, take the first observation as the moment a traveller starts watching.
2. At each subsequent observation, convert the model's forecast into a book-now or wait decision.
3. Compare the price paid under the simulated policy against booking immediately at the first observation, and against the retrospective minimum, which is the unreachable upper bound.
4. Report mean and median savings, the share of cases where the policy did worse than booking immediately, and the breakdown by city, review-score tier and lead-time bucket.

Reporting the loss cases alongside the savings matters. A policy that saves 20% on average while costing money in a third of cases is a different product from one that saves 15% with rare losses, and an average alone cannot distinguish them.

5.4 Data and System Quality

5.4.1 Data Quality

The collection pipeline is evaluated with the audit script already in use on the pilot:

- **Crawl success rate** per run, counting technical failures only. Sold-out nights, un-bookable properties and confirmed dead links are reported in their own categories.
- **Persistence completeness**: the share of successful items where candidate, parsed and saved option counts agree. A mismatch means silent data loss and is treated as a defect rather than as noise.
- **Series continuity**: the largest gap in calendar days for each (hotel, check-in date) series, since gaps invalidate the lag and rolling features computed near them.
- **Reference readiness**: the share of series with inventory that reached approved reference status, measured against the 80% coverage gate. The pilot reached 90.5%.
- **Missingness and anomaly rate** per field, per city and per property, tracked over time to catch a selector that degrades without breaking outright.
- **Duplicate and block counts**, both expected to stay at zero.

5.4.2 System Performance

- API response time at the 95th percentile under simulated concurrent load, target 200 ms.
- Data freshness: elapsed time from a crawl completing to its observations appearing in the dashboard, target one hour.
- Full retraining time on the accumulated dataset, target 30 minutes.
- Worker recovery: a worker killed mid-item releases its lease and the item is reclaimed and completed by the next worker, verified by fault injection.

5.5 Threats to Validity

- **Single-source dependence.** All rate data comes from Booking.com. Findings describe rate behaviour as published on that platform and do not generalise to rates a property publishes elsewhere. Section 1.3.3 explains the trade-off accepted here.

- **Collection window length.** A five-month window covers part of one annual cycle. Seasonal effects that repeat yearly, Tet above all, are represented by at most one occurrence, so any seasonal conclusion is reported as observed rather than as established.
- **Rates are not transactions.** The dataset records published rates, not bookings. Both simulations therefore measure signal quality against realised rates and cannot measure revenue or conversion.
- **Cohort selection.** The 355 properties were selected by city and by presence on Booking.com rather than by random sampling from a registry, so the cohort is not a statistically representative sample of Vietnamese accommodation. The composition is reported alongside the results.
- **Session-dependent inventory.** Booking.com may show different partner rates in different sessions, as the pilot observed directly. Audits use the artifact captured during the crawl as the reference, and a comparison against a later browser session is treated as a sanity check rather than as ground truth.

Bibliography

- [1] Vietnam National Administration of Tourism, “Vietnam Tourism Annual Report,” Ministry of Culture, Sports and Tourism, Hanoi, Vietnam. [Online]. Available: <https://vietnamtourism.gov.vn/> (accessed: Aug. 10, 2026).
- [2] General Statistics Office of Vietnam, “Statistical data on tourism and accommodation services.” [Online]. Available: <https://www.gso.gov.vn/> (accessed: Aug. 10, 2026).
- [3] Google, Temasek, and Bain & Company, “e-Conomy SEA 2023: The tipping point,” 2023. [Online]. Available: <https://economysea.withgoogle.com/> (accessed: Aug. 10, 2026).
- [4] G. Abrate, G. Fraquelli, and G. Viglia, “Dynamic pricing strategies: Evidence from European hotels,” *International Journal of Hospitality Management*, vol. 31, no. 1, pp. 160–168, 2012.
- [5] S. Ivanov and C. Zhechev, “Hotel revenue management: A critical literature review,” *Tourism: An International Interdisciplinary Journal*, vol. 60, no. 4, pp. 355–376, 2014.
- [6] Lighthouse (formerly OTA Insight), “Hotel rate intelligence and market insight platform.” [Online]. Available: <https://www.mylighthouse.com/> (accessed: Aug. 10, 2026).
- [7] RateGain Travel Technologies, “Rate intelligence and revenue management solutions.” [Online]. Available: <https://rategain.com/> (accessed: Aug. 10, 2026).
- [8] Hopper Inc., “Hopper: Book travel with price predictions.” [Online]. Available: <https://www.hopper.com/> (accessed: Aug. 10, 2026).
- [9] Q. Ye, R. Law, and B. Gu, “The impact of online user reviews on hotel room sales,” *International Journal of Hospitality Management*, vol. 28, no. 1, pp. 180–182, 2009.
- [10] Z. Zhang, Q. Ye, and R. Law, “Determinants of hotel room price: An exploration of travelers’ hierarchy of accommodation needs,” *International Journal of Contemporary Hospitality Management*, vol. 23, no. 7, pp. 972–981, 2011.
- [11] R. Law, G. Li, D. K. C. Fong, and X. Han, “Tourism demand forecasting: A deep learning approach,” *Annals of Tourism Research*, vol. 75, pp. 410–423, 2019.

- [12] Open-Meteo, “Open-Meteo: Free weather API.” [Online]. Available: <https://open-meteo.com/> (accessed: Aug. 10, 2026).
- [13] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, San Francisco, CA, USA, 2016, pp. 785–794.
- [14] L. Grinsztajn, E. Oyallon, and G. Varoquaux, “Why do tree-based models still outperform deep learning on typical tabular data?,” in *Advances in Neural Information Processing Systems 35, Datasets and Benchmarks Track*, New Orleans, LA, USA, 2022.
- [15] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [16] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [17] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems 30*, Long Beach, CA, USA, 2017, pp. 5998–6008.
- [18] A. Zeng, M. Chen, L. Zhang, and Q. Xu, “Are transformers effective for time series forecasting?,” in *Proc. 37th AAAI Conf. Artificial Intelligence*, Washington, DC, USA, 2023, pp. 11121–11128.
- [19] S. Ramírez, “FastAPI: Modern, fast, web framework for building APIs with Python.” [Online]. Available: <https://fastapi.tiangolo.com/> (accessed: Aug. 10, 2026).
- [20] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” in *Proc. 25th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, Anchorage, AK, USA, 2019, pp. 2623–2631.
- [21] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Advances in Neural Information Processing Systems 30*, Long Beach, CA, USA, 2017, pp. 4765–4774.